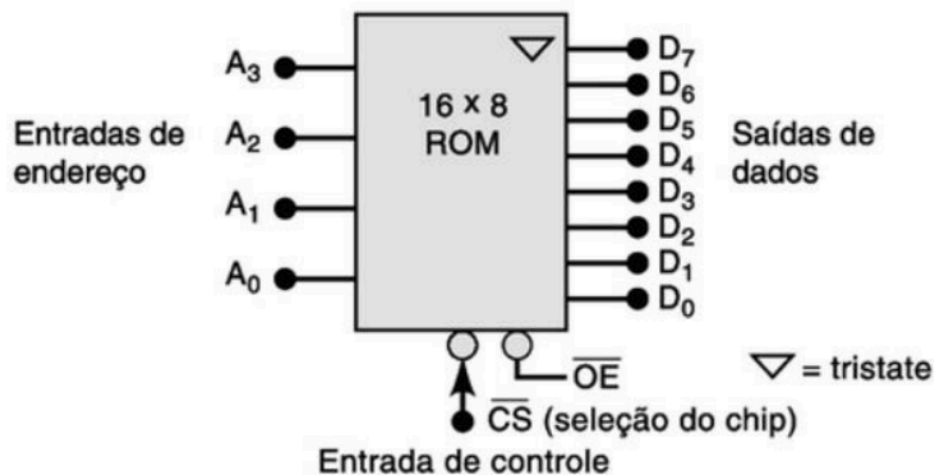


OPERAÇÃO E EXPANSÃO DE MEMÓRIAS



RW (ou WE) → define se vai **ler** ou **escrever**

- Quando está em **leitura**, a **entrada de dados fica em alta impedância**, porque o chip só precisa **enviar** dados pela saída.
- Quando está em **escrita**, a **saída fica em alta impedância**, porque o chip só precisa **receber** dados pela entrada.

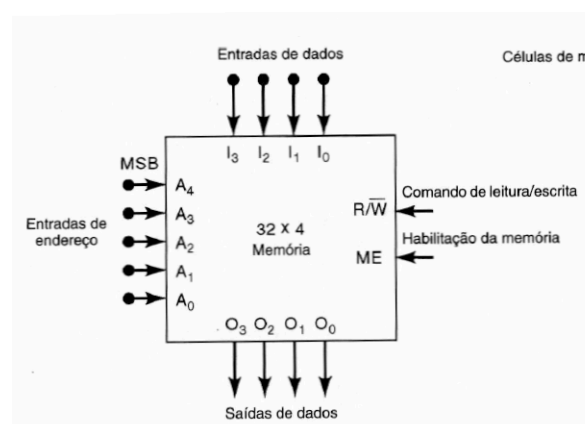
OE → habilita ou desabilita a saída da memória

CS (ou ME) → habilita ou desabilita o chip inteiro

O que acontece quando o OE é desabilitado?

- A saída da memória fica em **alta impedância**. Ou seja, fica "desconectada" eletricamente, sem enviar nada.

Por que algumas memórias não têm OE?



Porque **OE** é um sinal de controle extra que serve para dar mais flexibilidade ao sistema, mas **nem sempre é necessário**.

Em algumas memórias, principalmente **mais simples** ou **internamente otimizadas**, o controle de saída é feito **automaticamente** com base no estado de **RW/WE** e **CS**.

Nesses casos:

- Quando o chip **vai ler** ($RW = 1$ e CS ativo):
→ a **saída já é automaticamente ativada**
- Quando o chip **vai escrever** ($RW = 0$):
→ a **saída é desativada automaticamente** (alta impedância), porque ele só precisa **receber dados**

E quando o CS é desabilitado?

- A memória **inteira** fica em alta impedância. Não só a saída, mas todo o chip é eletricamente desligado do circuito

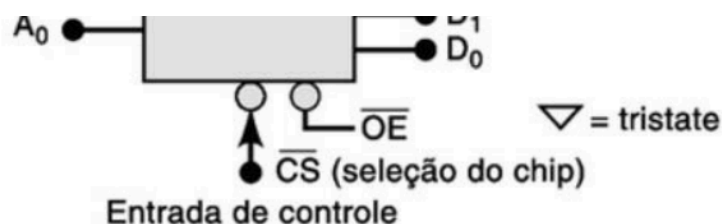
Obs: na maioria dos chips, **CS** e **OE** são **ativos em nível baixo** (ou seja, funcionam quando recebem 0).

MAS NEM SEMPRE!

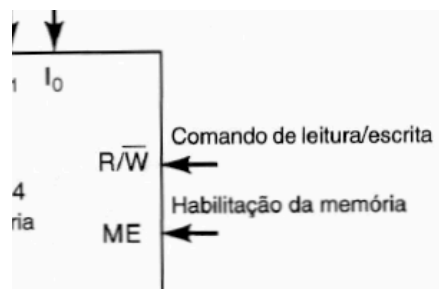
Pra ter certeza, analise o **diagrama** da memória.

- Se tiver uma **porta inversora (bolinha)** na entrada, é ativo em **nível baixo**. (barrada)
- Se **não tiver**, é ativo em **nível alto**.

Habilitada em nível baixo:



Habilitada em nível alto:



O mesmo vale para o **RW**: se o **W** estiver barrado, significa que a escrita é ativada em nível baixo, e vice-versa. O padrão é esse: **RW** ativa escrita em 0 e leitura em 1.

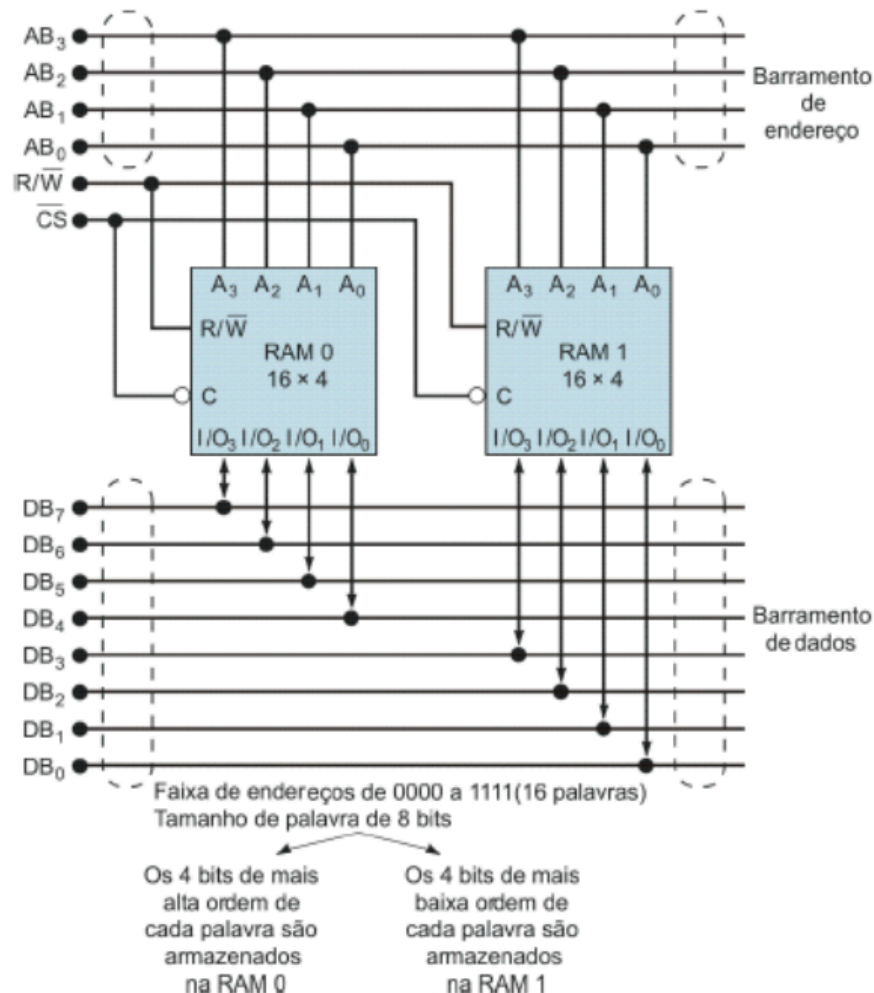
Com a opção de habilitar ou desabilitar a memória, conseguimos **combinar dois chips** de memória. Isso serve pra quê?

- **Aumentar o tamanho da palavra** (ex: de 8 bits para 16 bits)
- **Aumentar a quantidade de palavras** que o sistema pode armazenar (ex: de 1k para 2k palavras)

Como fazer isso?

1. Para expandir o tamanho da palavra (por exemplo, de 4 bits para 8 bits):

- Conecte os **endereços iguais** em ambos os chips.
- Um chip vai armazenar os **4 bits mais baixos** (bits 0 a 3), e o outro os **mais altos** (bits 4 a 7).
- As saídas dos dois chips se juntam para formar a palavra de 8 bits.



2. **Para expandir a quantidade de palavras** (por exemplo, dobrar a memória(capacidade) de 16 para 32):

- Use um **bit extra de endereço** para ativar um chip por vez, via **CS**.
- Exemplo: se AB_4 for 0, ativa o primeiro chip. Se AB_4 for 1, ativa o segundo (lembrando que deve ser colocado um inversor na entrada do segundo chip).
- As saídas dos chips são ligadas no mesmo barramento, mas como só um está ativo por vez, não há conflito.

